



Speed Optimized Implementation for Three Lift Controller

Y.JANAKIRAM¹, U.PRADEEP KUMAR²

¹PG Scholar, Dept of ECE, Pydah College of Engineering & Technology, Visakhapatnam, Andhrapradesh-INDIA,
E-mail: ram.rjy1@gmail.com.

²Asst Prof, Dept of ECE, Pydah College of Engineering & Technology, Visakhapatnam, Andhrapradesh –INDIA,
E-mail: pradeepkumar.urity@gmail.com.

Abstract: The high growth of the semiconductor industry over the past two decades has put Very Large Scale Integration in demand all over the world. The basics of digital logic theory and techniques are easily understood by the design based on VLSI technology. These are the core fundamentals of the fast, high-speed complex digital circuits. In modern life, elevators have become an integral part of any public or commercial complex. It does not only ease the faster movement between any two floors and provide a way for movement of disabled, but has also become a status symbol. Elevators, as usually been called as “cars” which work on a gearless traction system in which the movement of the elevator is controlled by several steel hoist ropes and a counter-weight. The weight of the car and counterweight provides sufficient traction between the sheaves and the hoist ropes so that the sheaves can grip the hoist ropes and move and hold the car without excessive slipping. The machinery to drive the elevator is located in a machine room usually directly above the elevator hoist way. To feed electricity to the car and receive electrical signals from it, a multi-wire electrical cable connects the machine room to the car. Elevator controller controls the entire operation of the three elevator system. The proximity sensors located to sense the positions of the cars, provide the current state storing it in register. The obstruction sensors provide the status of obstruction. The elevator controller also reads the requests, if any, from any of the request positions through the flip-flops. If the door of any elevator is open, the timer signals from the elevator keep the controller informed of being busy. The control state machine receives all these signals. It is programmed to the algorithm by which it should control the system. The FSM then generates control signals for the next position and movement of the elevators. Elevators on receiving the signal address to the request, as been asked by the controller. In this work, the real-time three elevator controller will be modeled with Verilog HDL code using Finite-State machine (FSM) model to achieve the logic in optimized way.

Keywords: Elevator, Finite-State Machine, Verilog , Xilinx ,Modelsim

I. INTRODUCTION

An elevator or lift is a kind of transport device used to move people between building floors. Whenever a passenger presses the call button for an elevator, a computer receives the request and logs it for future reference. There are actually two sets of doors, which allow passengers safely to exit and enter the lift. One set of doors remains shut until an elevator car's presence is detected and the elevator's computer controls the other door. Once both doors are open, passengers should leave quickly to allow new passengers to board and more calls to be answered. Elevator doors also contain motion detectors and other presence-sensing devices to keep doors from trapping passengers. Another issue want to be considered is the weight capacity. Overload in elevator will lead to accident and using the load sensor can control it. In modern life, elevators have become an integral part of any public or commercial complex. It does not only ease the faster movement between any two floors and provide a way for movement of disabled, but has also become a status symbol. Elevators work on a gearless traction system in which the movement of the elevator is

controlled by several steel hoist ropes and a counter-weight. The weight of the car and counterweight provides sufficient traction between the sheaves and the hoist ropes so that the sheaves can grip the hoist ropes and move and hold the car without excessive slipping. The machinery to drive the elevator is located in a machine room usually directly above the elevator hoist way. To feed electricity to the car and receive electrical signals from it, a multi-wire electrical cable connects the machine room to the car. In this chapter, design of three elevator controller is made using Verilog HDL and is verified by using the MODEL SIM and XILINX ISE tool.

II. BASIC PRINCIPLE

Elevators themselves are simple devices, and the basic lifting systems have not changed much in over 50 years. The control systems, however, have changed substantially to improve safety and speed of operation. Elevators are designed for a specific building, taking into account such factors as the height of the building, the number of people

traveling to each floor, and the expected periods of high usage.

Controllers can also be programmed to respond differently at different times of the day. For example, the elevator controller in a busy office building will receive a preponderance of calls from the ground floor in the morning, when workers are arriving and need to go to their workplaces on the upper floors. In that case, the controller will be programmed to send all unassigned cars to the ground floor, rather than have them return to a home floor in their sector. Later in the day, a different set of

instructions can be used to send unassigned elevators to different sectors, since passengers leaving the building will be much more evenly distributed among the floors than in the morning.

A. Survey on Elevator Control

Elevators, as usually been called as “cars” come in three or quad groups. The coordination of a group of cars in elevator systems for satisfying the demands of passengers is called the **Elevator Group Control**. The Elevator controller provides this control by monitoring

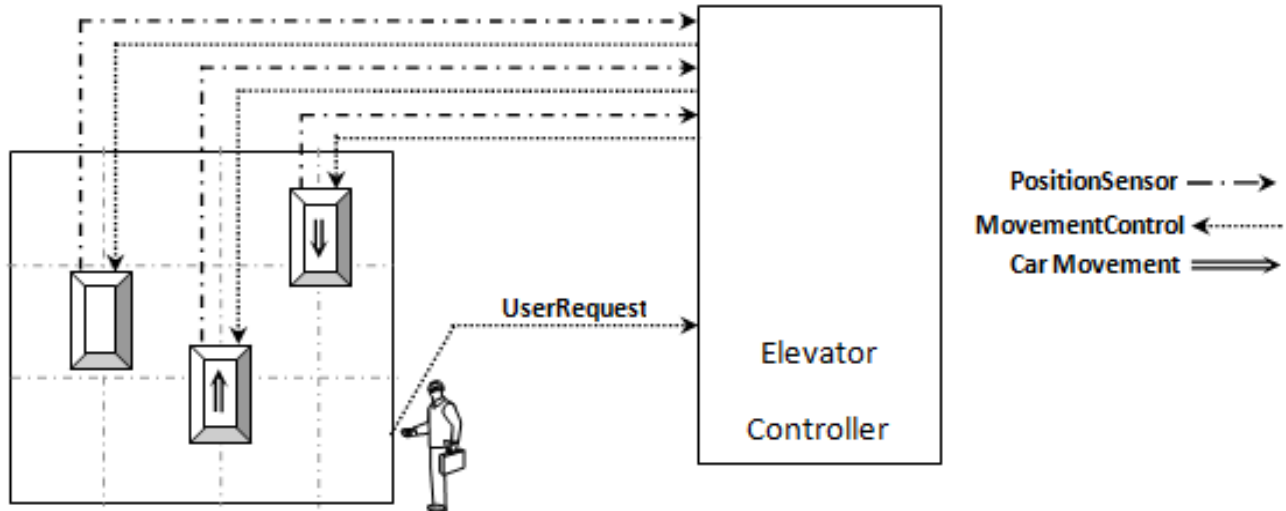


Figure1. Elevator Group Control

the position of all the elevators. When a request comes, the controller checks the position of the request and assigns the elevator nearest to it, to address the request. Generally, it stores an algorithm to take the action according to the request position, and positions of the cars. It has proximity switches to sense the position of the elevators. Amongst the many factors included in the selection of the car, wait time of the user is dominant.

B. Elevator Controller & it's Working

Elevator controller controls the entire operation of the three elevator system. The proximity sensors located to sense the positions of the cars, provide the current state storing it in register. The obstruction sensors provide the status of obstruction. The elevator controller also reads the requests, if any, from any of the request positions through the flip-flops.

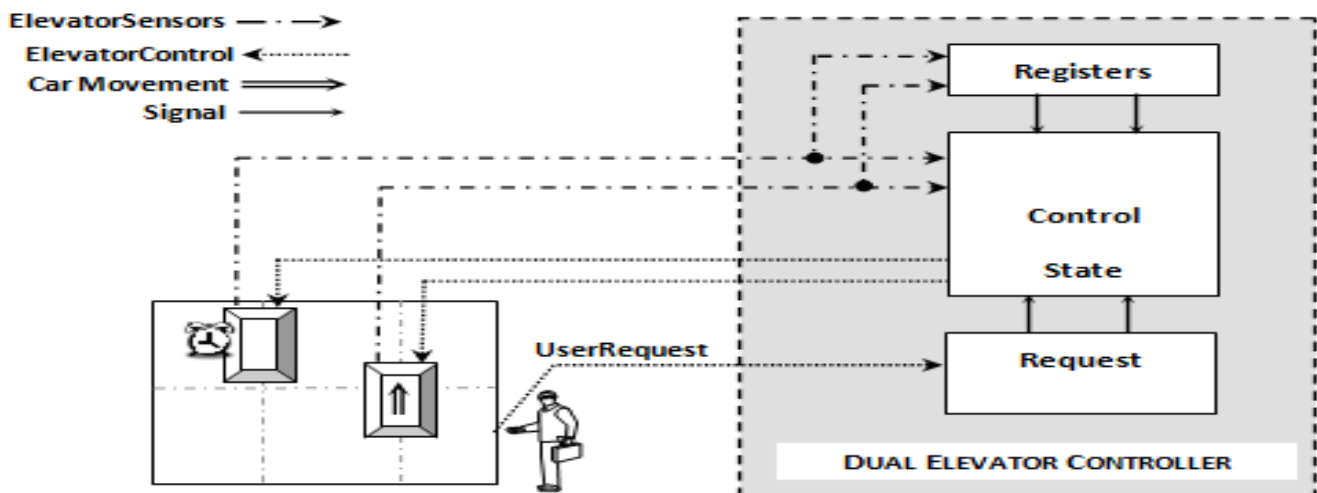


Figure2. Block diagram for Three Elevator Controller

If the door of any elevator is open, the timer signals from the elevator keep the controller informed of being busy. The control state machine receives all these signals. It is programmed to the algorithm by which it should control the system. The CSM, then generates control signals for the next position and movement of the elevators. Elevators on receiving the signal address to the request, as been asked by the controller.

III. DESIGNING OF ELEVATOR

The three elevator controller designing in HDL has been a great challenge. The Moore model did simplify the approach, but in order to operate the system in optimized way and accept user request by the nearest elevator, highlighted the complexities involved in it. The elevator algorithm respects the constraints defined and works in alignment to the assumptions made. The timer implementation as a part of the code remained an unresolved issue, as the coding should also have been synthesizable. The other issues that could not be addressed were to store the request and provide to the elevator at a later stage if the elevator is in busy state at that time. As the elevator system in for only two floors, the same could also be automated that once user gets into the lift, he does not need to request for the destination.

In general, the elevator controller has to control a larger group in a multi-floor building. Then, the capabilities of designing the algorithm of the designer decide the functionality of the elevator system. The building has two elevators that move between two floors. The first floor has a single UP button and the second floor has a single DOWN button. Inside each elevator has also two buttons, UP and Down. Normally an elevator will move when instructed to do so by the control, opening the door for 20 second. After the waiting of 20 second the door might be closed. If any obstruction detected, the door will open. The input of the system includes sensors that indicate whether the elevator is at the given floor and if a passenger moving ON or OFF on the elevator is obstructing its doors.

A. Assumptions for the Elevator System

The Three Elevator System does face some conflict in the operation that which car should take the request when both are at the same positions. So, some assumptions are implemented in the elevator algorithm. The assumptions considered are:

1. **Elevator Priority:** Elevators are prioritized for the requests. The elevator1 has a priority for the first floor request and the elevator2 for the request from second floor.
2. **Default State:** Elevator1 on first floor with closed door and the elevator2 at second floor with closed door. This default position provides quick response to the request coming at any of the two floors.
3. **Closing the Elevator Door:** Door of the elevator close after some time duration, defined by the timer. By default the timer should be 0 and when to close the door, it should be 1. The system also checks for

any obstruction if present between the doors. Both, the timer and obstructions, are implemented using switch.

Thus, to close the door the door of an elevator, the respective timer needs to be triggered to high state and the obstruction should be 0.

B. WORKING

Now, let us consider the basic and general block diagram of three elevator controller which is shown below. In this general block diagram, there are two main blocks are available. One is controller and other is door operator. A sensor is connected between these two main blocks, so that if there any sensor signal goes high, the operation on both elevators 1 and 2 will be carried out.

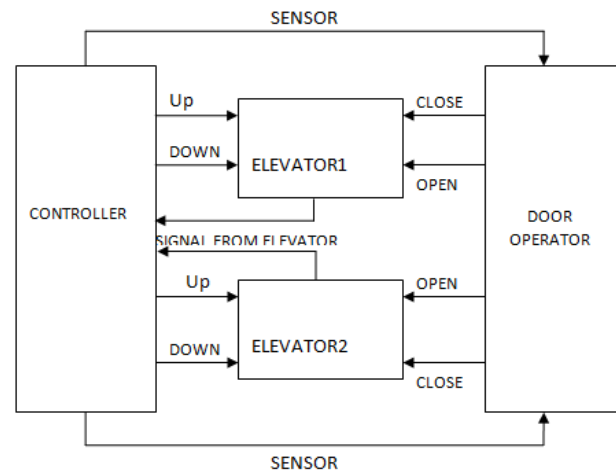


Figure3. Basic Block Diagram of Elevator Controller

Elevator has up button and close button inside it and outside the elevator there is up button in first floor and down button in second floor. Door operator will make the elevator door to open or close with respect to the request made. When there is any request signal from elevator will go to the controller which in turn gives the signal to elevator with respect to request signal. For example, when up button is pressed inside elevator 1 will give request to controller and from controller the signal will go to elevator 1 through up button. Similarly for elevator 2 also it works. The sensor signal will sense the obstruction and timer, so that the signal from controller will be send to door operator and hence the door operator will control the elevator according to the request. Controller will have flip-flops to store the previous state of elevator and comparator to check the position of the elevator with respect to the request signal given. Timer circuit will provide the delay of 20 second when the elevator door is in open condition. These conditions will be applied for both the elevators.

C. INPUTS AND OUTPUTS

The inputs used in designing elevator controller are as follows:

- Clock
- Reset
- Request from First Floor

- Request from Second Floor
- Request made inside Elevator1 from First to Second floor
- Request made inside Elevator1 from Second to First floor
- Request made inside Elevator2 from First to Second floor
- Request made inside Elevator2 from Second to First floor
- Obstruction Sensor used in Elevator1
- Obstruction Sensor used in Elevator2
- Timer used to close the Elevator1 door
- Timer used to close the Elevator2 door

Table1. Inputs for the Elevator System

S.N.	Input	Code
1.	Request from first floor	req_FF
2.	Request from second floor	req_SF
3.	Request from Elevator1 to move from first floor to second floor	req_E1_F2S
4.	Request from Elevator1 to move from second floor to first floor	req_E1_S2F
5.	Request from Elevator2 to move from first floor to second floor	req_E2_F2S
6.	Request from Elevator2 to move from second floor to first floor	req_E2_S2F
7.	Obstructions for Elevator1	obs_sensor1
8.	Obstructions for Elevator2	obs_sensor2
9.	Time out signal for closing door of Elevator1	open_timer1
10.	Time out signal for closing door of Elevator2	open_timer2

The outputs used in elevator controller design are as follows:

- E1_pos – Position (Present State) of Elevator1
- E2_pos - Position (Present State) of Elevator2

E1_Next_Pos and E2_Next_Pos (Next State) are taken as reg in order to store the previous position of elevators.

D. ASSUMPTIONS

- ❖ Elevator E1 has priority for First Floor
- ❖ Elevator E2 has priority for Second Floor
- ❖ At the reset state, elevators are should be in its priority floor. The obstruction sensor is set to HIGH when there is any obstruction present in between the elevator doors and is set to LOW when no obstruction.
- ❖ Timer is considered as switch. So when the elevator door gets open, timer will set to LOW and after some time period, if no obstruction, timer set to HIGH after which door get close.
- ❖ Only one request will be executed during one cycle.
 - ❖ Totally there are 36 possible states present when considering both the elevators and we have considered 6 states for single elevator.

D. Finite State Machine

A finite-state machine (FSM) is a mathematical abstraction sometimes used to design digital logic or computer programs. It is a behavior model composed of a finite number of states, transitions between those states, and actions. The finite-state machine is similar to a flow graph in which one can inspect the way logic runs when certain conditions are met. It has finite internal memory, an input feature (reading symbols in a sequence, one at a time without going backward), and an output feature, which may be in the form of a user interface, once the model is implemented. The number and names of the states typically depend on the different possible states of the memory, e.g. if the memory is three bits long, there are 8 possible states. The state that reads the first symbol of the input sequence is called the start state and the state which signifies the successful operation of the machine is called the accept state. Every state may lead to a different one depending on the next input symbol and output can be provided during the transition. State machines can also have probabilistic transitions, fuzzy states and other oddities.

Depending on the number of states an input can lead to, finite-state machines can be divided into two categories: deterministic and nondeterministic. The nondeterministic machine may have any number of following states for a given input symbol or none at all, whereas the deterministic machine has exactly one following state for a symbol. An important property of finite-state machines is that every nondeterministic machine has a corresponding deterministic one that accepts the same words. This is not true for pushdown automata. Finite-state machines can solve a large number of problems, among which electronic design automation, communication protocol design, parsing and other engineering applications.

Mealy and Moore models are the basic models of state machines. A state machine which uses only Entry Actions, so that its output depends on the state, is called a Moore model. A state machine which uses only Input Actions, so that the output depends on the state and also on inputs, is called a Mealy model. The models selected will influence a design but there are no general indications as to which model is better. Choice of a model depends on the application, execution means (for instance, hardware systems are usually best realized as Moore models) and personal preferences of a designer or programmer. In practice, mixed models are often used with several action types.

A Mealy machine can have its output depend on both input and state. Thus, if we ignore the state, we should be able to convert a Moore machine to a Mealy machine. It's not so easy to see that you can convert an arbitrary Mealy machine to a Moore machine. It turns out that the two machines are equivalent. What does that mean? It means that given a Moore machine, you can create a Mealy machine, such that if both machines are fed the same

sequence of inputs, they will both produce the same sequence of outputs. You can also convert from a Mealy machine to its equivalent Moore machine, and again generate the same outputs given the same sequence of inputs. Actually, to be precise we must ignore one fact about Moore machines. Moore machines generate output even if no input has been read in. So, if you ignore this initial output of the Moore machine, you can convert between one machine and the other. The actual algorithm is beyond the scope of the course. However, the basic idea of converting a Mealy machine to a Moore machine is to increase the number of states. Roughly speaking, if you have a Mealy machine with N states, and there are k bits of input, you may need up to $2^k N$ states in the equivalent Moore machine. Effectively, the new states record information about how that state was reached.

E. STATE DIAGRAM

The figure4 shows the state diagram for single elevator controller. Initially, the elevator is at its priority floor with closed doors. As and when the request comes, the controller assigns it to the elevator, and w.r.t the current status, obstruction and timer. This controller state diagram can be applied to three elevators, by duplicating the same and defining the input signals w.r.t the elevators. The request signal from the floors, remain common to both. On reset, each of the elevator rest at their respective priority floors. Whenever the elevator is in moving state, it does not receive any request and at the next clock cycle opens the door at the destination floor. When doors of an elevator are open, it gets closed only when the related signal has gone low, the timer out is high to reflect the open duration over and the obstruction signal is low.

When both the elevators are at the same state, the elevator with priority for the floor from where the request is raised, changes state. In rest cases, the elevator that would take less time to service the request is assigned to it.

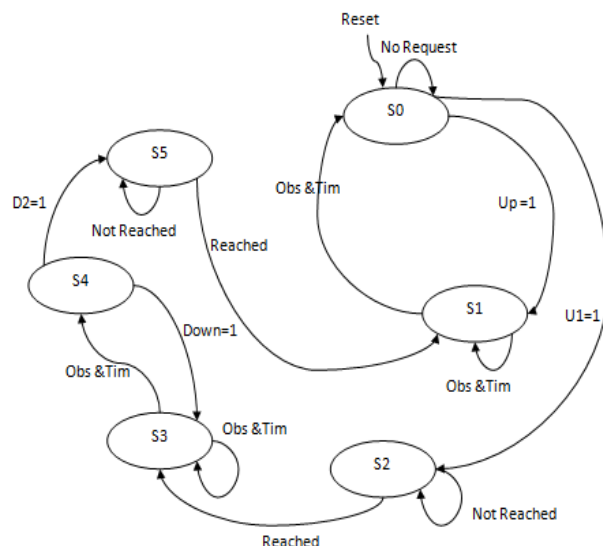


Figure4. State Diagram for Elevator Controller

Examples: Both the elevators are at first floor with closed doors. To address any request, the priority defined for the elevators for the floor of request would decide the elevator to change state. When there is a request from first floor, elevator1 would open the door to address it. Suppose, both the elevators at second floor, with elevator1 with open door and elevator2 with closed door and a request arises from first floor. Although elevator1 has priority for the floor, but because elevator2 is available to address the request, the request is assigned to the elevator2.

The Moore machine of single elevator controller is drawn and shown in fig4. Single elevator is considered for drawing state diagram and the same conditions will be applied for second elevator also. With respect to the position sensors, we can activate either elevator1 or elevator2, corresponding to the request made. This is one of the ways to make the elevator to work.

Table2. State Description

S0	First Floor _ Close door
S1	First Floor _ Open door
S2	Moving _ First to Second
S3	Second Floor _ Open door
S4	Second Floor _ Close door
S5	Moving _ Second to First

Table3. Input Description

Up	Up button in first floor
Down	Down button in second floor
U1	Moving _ First to Second
U2	Second Floor _ Close door
Obs	Obstruction
Tim	Timer
D2	Moving _ Second to First

Above shows the description of each state shown in state diagram. S0 is the initial state in which the elevator will stand at its priority floor. If no request, elevator will be in initial state. Once the up button in first floor is pressed, elevator door will get opened is shown as transition S1. It checks for obstruction and timer in order to close the door and if there is any obstruction is present, and then the door will be in open condition only. When D1 is pressed elevator will come to state S1 (Open Door) again and U1 will leads to state S2 (Move from 1 to 2). After reaching second floor, elevator door should get open represented by state S3. If no obstruction and timer is set to HIGH, signal goes to close door state S4.

When the down button in second floor is pressed, door goes to open state S3 from which obstruction and timer is checked and close the door. After door gets closed, D1

inside the elevator will leads to moving from 2 to 1 state S5. The elevator door goes open state S1when it reaches first floor. At last it checks for obstruction and timer to come back to initial state S0. Thus the elevator completes its cycle by satisfying all the possible conditions. Same will be followed for the second elevator with respect to its priority floor and input request.

IV. TEST CASE ANALYSIS

For verification of the HDL coding done for the three elevator controller, a test bench is prepared. The same is available in Appendix 2. It check some of the typical cases w.r.t. the different positions of the elevators and the input requests. A few of them are discussed below; Test analysis will be giving the clear view on working or process of three elevators that designed. The waveform can be shown using MODELSIM Tool. Some cases are considered for the test and they are analyzed with respect to given inputs. Hence the analyzed report will provide the clear working of three elevator controller.

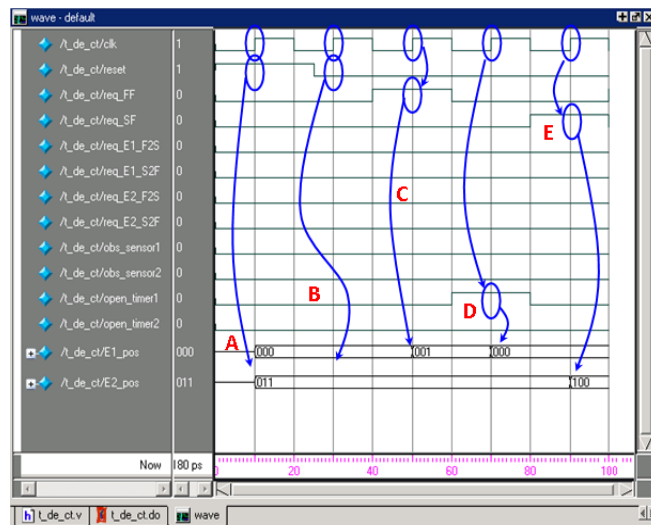


Figure5. Waveforms for test case 1

Test Case 1: The test case 1 waveforms are displayed in the figure 2.4. The circuit is positive edge triggered, and here are five sub-cases:

- A - As the reset signal is high, the elevator position gets reset to the closed door position at the respective priority floor.
- B - When reset signal is low, the previous state is held by elevators.
- C - When there is call from first floor, the elevator1 opens the door.
- D - After the timer1 is timed out, elevator1 closes the door at first floor.
- E - When there is a request from second floor, the elevator2 opens the door.

Test Case 2: The test case2 waveforms are displayed in the figure6. This case again starts from the initializing states by reset signal. Here are six sub-cases:

- F - Again the reset signal to force elevators at default position.

G - The previous state is held by elevators, when reset signal is low.

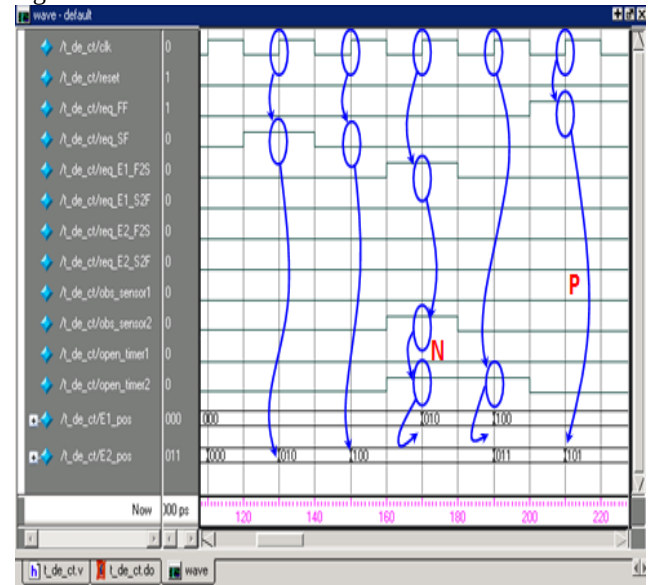


Figure6. Wave forms for test case 2

H - When request from elevator2 to go down, the elevator2 moves to first floor.

I - After reaching first floor, elevator2 opens the door, even when no input signal.

J - When there is a request from first floor, and the elevator2 is with open door, no change of state occurs.

K - When the timer2 is timed out, the elevator2 closes the door.

Test Case 3: The test case2 waveforms are displayed in the figure7. Both the elevators are at first floor with closed door. Here are five sub-cases:

- L - When there is a request from second floor, the elevator2 starts moving up.
- M - After reaching second floor, it opens the door, even when no input signals.

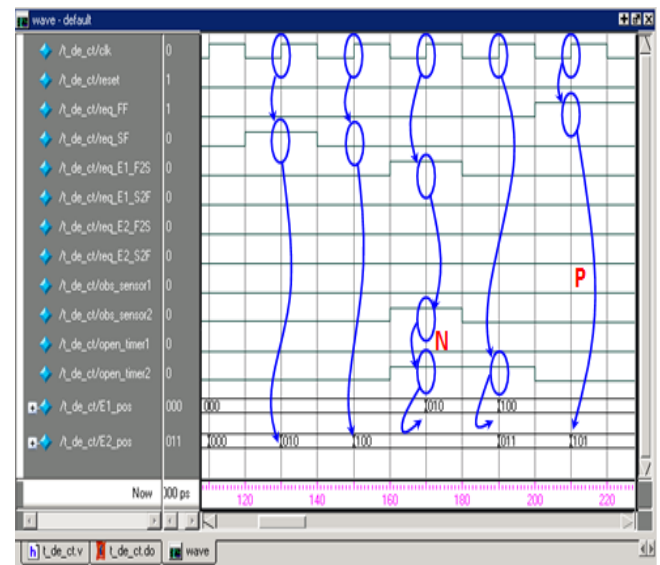


Figure7. Wave forms for test case 3

Speed Optimized Implementation for Three Lift Controller

N - When request from elevator1 to go up, the elevator1 starts moving to second floor. The timer2 has timed out but as the obstruction2 is high, the levator2 remains in the open door state at second floor.

Q - After reaching second floor, elevator1 opens the door, even when no input signals. In addition, as the timer2 is high and the obstruction2 has gone low, the elevator2 closes the door.

P - Although both the elevators are at second floor, but the door of elevator being open, the elevator2 comes out to be nearer to address the request from first floor. Though, elevator2 has low priority for first floor it moves downwards.

Q - Although not mentioned in the figure, but the elevator2 has to open the doors at first floor in completion of the request it is addressing.

From the above test cases, it is observed that the elevator controller follows the principle of assigning the request to the elevator that would keep the user in wait state for lesser time. It also considers the priorities define in the algorithm for its working.

ANALYSIS-1: In the Test Analysis Report - 1, few cases are considered. At the time of reset condition both

the elevators are should be in their priority floor. When the passenger in first floor press the up button inside elevator1, should leads to the second floor (010) and elevator1 door gets open (100). Timer1 is set high in order to close the door provided with no obstruction. When we consider the request coming from second floor, elevator2 in the second floor should accept the request and opens the door (100). Once door gets open, obstruction and timer signal will get activated. When obstruction is LOW and timer2 is HIGH, door should get closed (011). After closing, by pressing down button elevator2 will move from second to first floor (101) and get the door open (001) by reaching the destination. Now again it will wait for obstruction and timer2 signal and with respect to that door may get closed or opened.

The above cases we considered are represented in waveform as shown below. Test analysis will be giving the clear view on working or process of three elevators that designed. The waveform can be shown using MODEL SIM Tool. Some cases are considered for the test and they are analyzed with respect to given inputs. Hence the analyzed report will provide the clear working of three elevator controller.

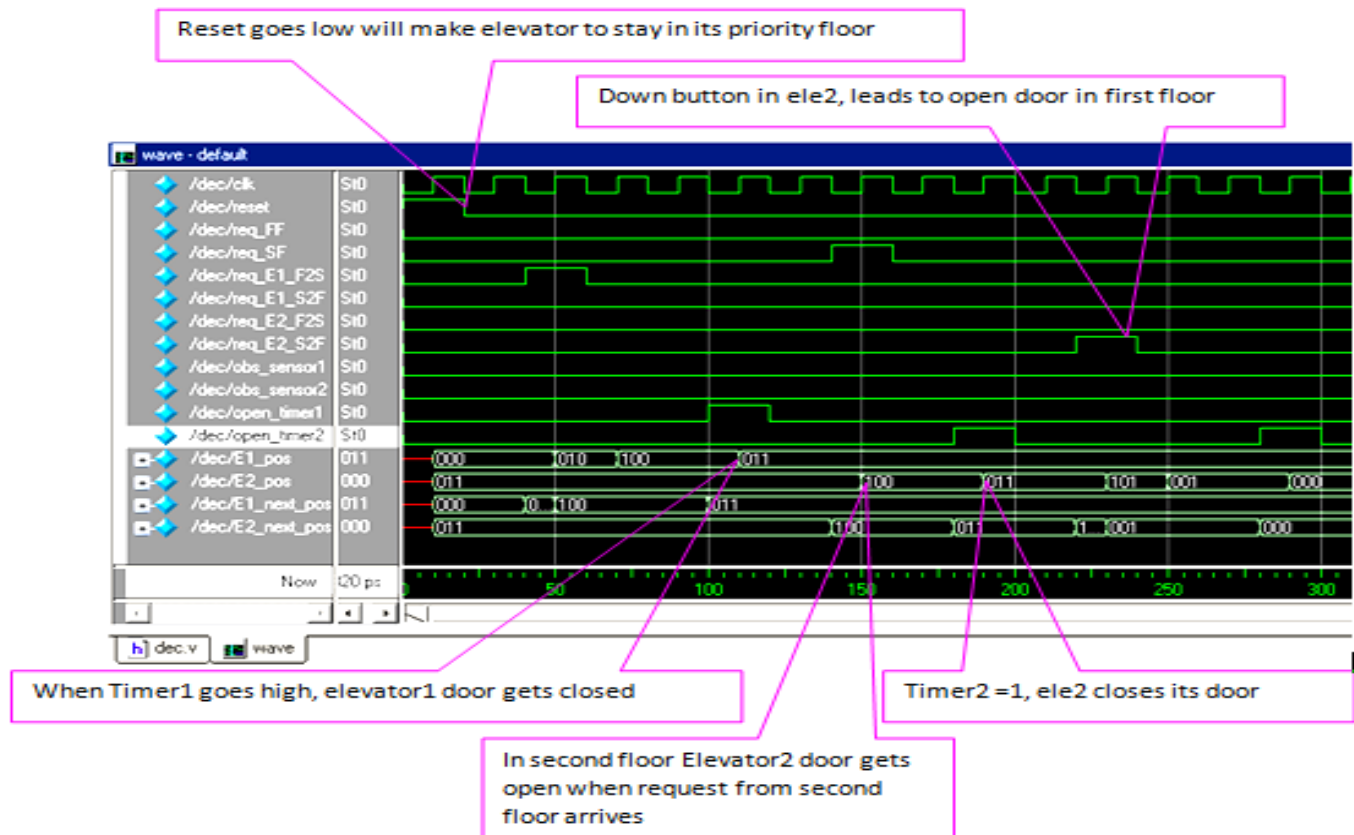


Figure8. Test Analysis Result - 1

ANALYSIS-2: In the Test Analysis Report – 3 shows obstruction and timer operation. Let us consider the request from first floor which will leads to open door of elevator1 (001). When the obstruction sensor1 set to high, even though timer set to high, door will not get close.

Once the obstruction gets cleared, elevator door will get closed (000) provided with timer to high signal. The obstruction and timer operation is clearly represented below.

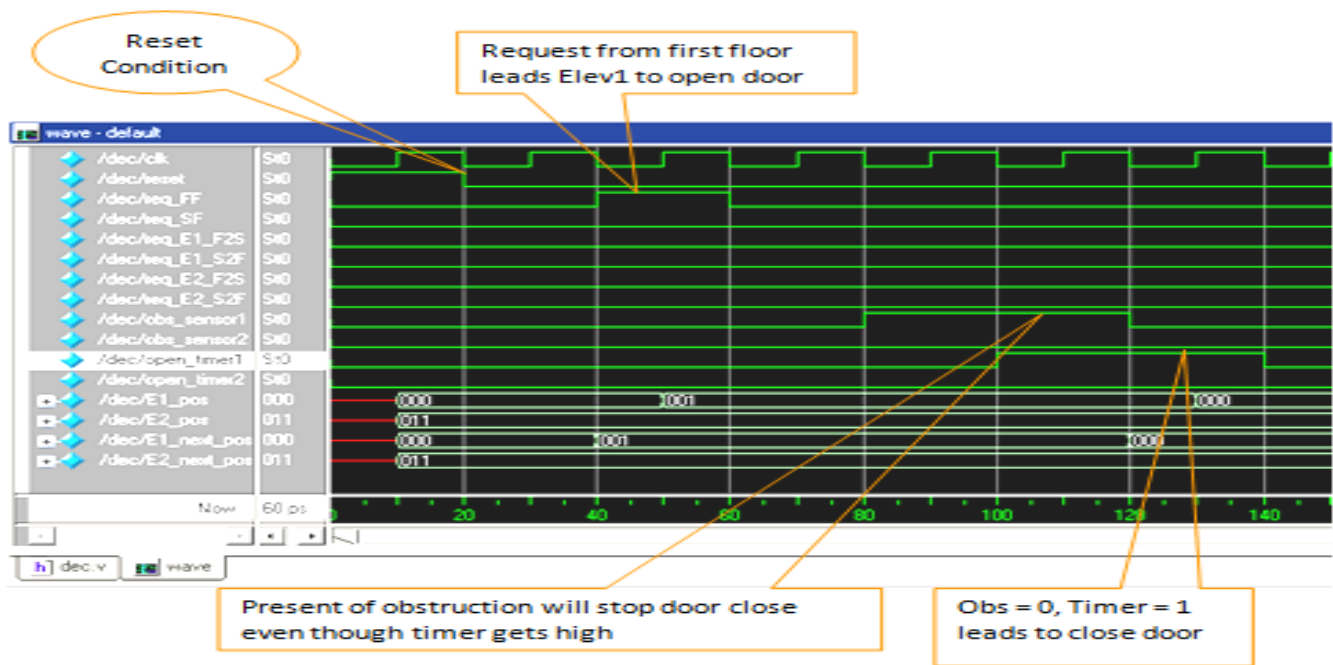


Figure9. Test Analysis Result – 2

ANALYSIS –3: In the Test Analysis Report–3, represents improper or wrong request given by the user. When the user presses down button inside the elevator that standing in first floor implies the improper request. In this design, when any improper operations made will leads to

the opening of elevator door. Then it will check for obstruction and timer in order to close the door which is already represented in analysis–2. User standing inside elevator2 in second floor presses the up button will leads to open door of elevator2 (100). This is clearly represented in waveform that shown below.

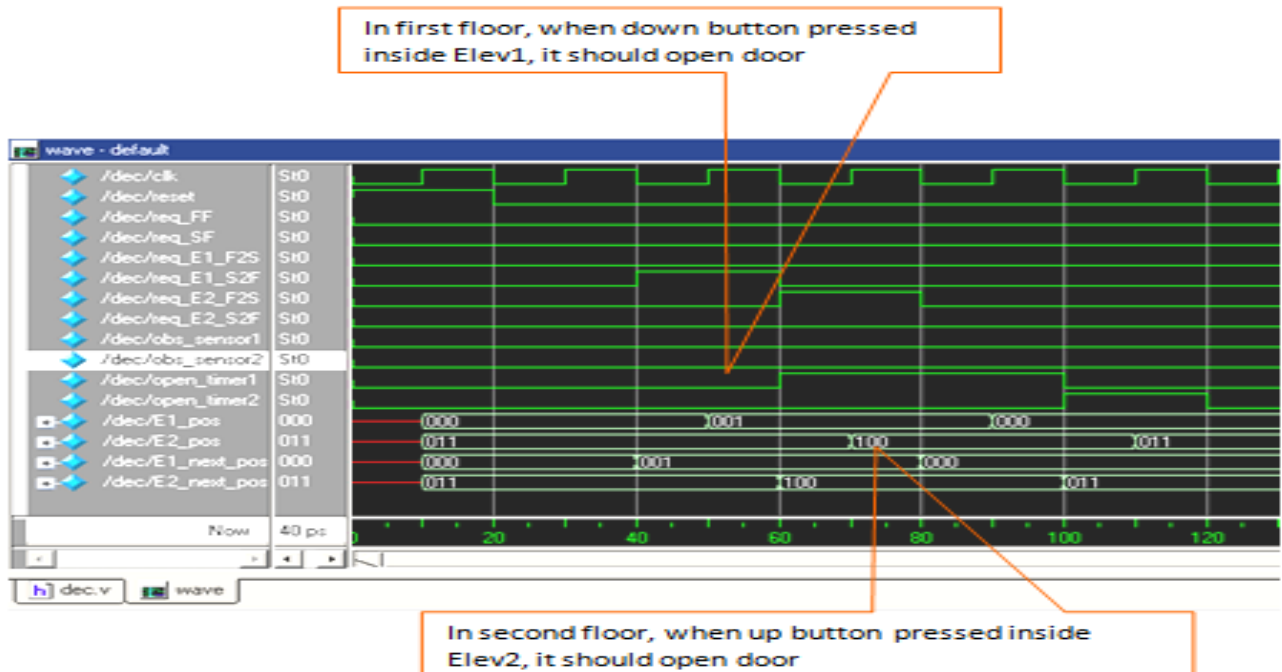


Figure10. Test Analysis Result – 3

ANALYSIS –4: In the Test Analysis Report–4, represents improper or wrong request given by the user. When the user presses down button inside the elevator that standing in first floor implies the improper request. In this design, when any improper operations made will leads to

the opening of elevator door. Then it will check for obstruction and timer in order to close the door which is already represented in analysis – 3. User standing inside elevator2 in second floor presses the up button will leads to open door of elevator2 (100).

V. CONCLUSION

In this chapter, the design of a Three Elevator Controller is made using Verilog HDL and is verified by using MODELSIM Tool. This tool helps us to identify whether the design working properly or not. The design made, have delay code which will be used while implementing it in FPGA board. Only one request will be carried out in one clock cycle period. The timer is used to provide delay when closing the elevator door. But in our design, timer is working properly in in XILINX and MODELSIM Tool. So we consider the timer operation is carried out by using switch. In future implementation, when all the elevators are moving in same direction should also response to the request made by the user. For example, when all the elevators are moving up, at that time request made from first floor should be responded.

VI. REFERENCES

- [1] SHEN Yanchun, ZHANG Yinpu, and WU Zheng, "Design of the elevator controller based on FPGA," Science and Technology Innovation Herald, pp. 94-95, N0.32, 2007.
- [2] HE Jingkai, WANG Ruichun, and WAN Xueyuan, "System Design of the elevator controller based on FPGA," Journal of Shenzhen Institute of Information Technology, Vol.8, pp. 57-60, June 2010.
- [3] CHEN Zhaofang, "Elevator controll system based on FPGA," Fujian Computer, pp. 164, 2010.
- [4] CHEN Huiping, WANG Zhangrui, MA Qiqiang, and HUANG Xuekun, "Implementation of three-lift controller based on FPGA," INSTRUMENTATION CUSTION, Vol;.14, pp. 72-73, 2007.
- [5] SU Xinhong, and ZHANG Haiyan, "Design of smart elevator system based on FPGA," SCIENCE & TECHNOLOGY INFORMATION, Vol. 26, pp. 103,2010.
- [6] GAO Yinghui, HOU Zhongxia, and YANG Chenlin, "Sesign of FPGA-based automatic elevator controller," Journal of Liaoning Technical University, Vol. 25, pp. 242-243, April 2007.